DASHBOARD OPERATING GUIDE

NYC 311 / R Shiny Dashboard

Opened and closed trends, shared model forecasts, and monthly staffing estimates

Prepared 2026-10-24 | Detailed user, data, model, and operational reference

Local URL	http://localhost:3838/
Launch	Rscript src/app.R
Runtime	Shiny + Plotly + Reticulate + configured Python environment
Main audience	R users and supervisors reviewing monthly trends and cost scenarios

Important data and decision boundary

NYC 311 service-request records are not equivalent to phone-call arrivals, agent schedules, or payroll. Forecast conversions and labor inputs are scenarios unless calibrated against authorized operational records. Dashboard output is planning support and requires human review.

1. Purpose and page layout

The R Shiny application provides a second interface to the monthly NYC 311 analytics workflow. Its trend panel displays opened or closed requests by date and borough. Its forecast panel projects monthly request volume using the same Python forecasting backend used by the Python Dash app. A third panel estimates call-taker needs and labor costs.

The page is organized into a trend view, monthly forecast view, and staffing optimization view. Each section has its own data or model controls and an action button. The bottom footer identifies the latest request date loaded at startup.

2. Start-up and environment

Launch from the repository root with Rscript src/app.R. The app listens on port 3838 by default and binds to 0.0.0.0. Set SHINY_PORT or SHINY_HOST to change those values. The application requires Shiny, Plotly, and Reticulate.

Reticulate selects the Python executable in NYC311_PYTHON, then RETICULATE_PYTHON, then the repository .venv/bin/python, with python3 as a final executable lookup. The chosen environment must have the project's Python requirements installed, including pandas, PyArrow, forecasting libraries, and MLflow.

NYC311_DATA_ROOT points to the data root containing dashboard_data.parquet. The app reads that Parquet data through Python, then calls the shared Python data refresh and forecasting functions. A live refresh failure prevents the app from falsely presenting stale data as current.

- Check the selected Python interpreter with NYC311_PYTHON if Reticulate reports import or package errors.
- Confirm the Parquet path and read permissions when start-up stops with a missing data error.
- The footer's current-through date reflects the refreshed request data loaded for this session.

3. Opened and closed request history

Choose OpenedSR or ClosedSR and a borough, then press the view action. Opened records are grouped by created date; closed records are grouped by close date. Borough filtering is applied before the time series is drawn.

Missing dates in the monthly forecast series are filled with zero. The history plot is daily and interactive; the forecast input is monthly. Opened and closed totals are not a matched cohort analysis and should not be interpreted as a service-level measure.

4. Forecasts and MLflow tracking

The forecast controls select the target data type, borough, number of forecast months, and one of Best, Holt-Winters, AutoARIMA, or LSTM. These choices call the shared src/forecasting.py backend, so model requirements and output semantics are consistent with the Python Dash application.

Best compares candidates on a chronological holdout using MAE, selects the lowest-error successful model, refits it using the full history, and returns a point forecast and widening approximate bounds. Candidate scores, model parameters, and selected run metadata are tracked in MLflow. The app displays the chosen model, holdout MAE, and run ID.

At least 24 monthly observations are required for automatic model selection. Individual model failures are logged; if no candidate can be evaluated, the forecast action reports an error rather than inventing an estimate.

5. Staffing and labor-cost planning

Press Optimize staffing to forecast opened requests for the selected borough and horizon, convert those requests to a call scenario, and calculate staffing for each projected month. The Erlang-C engine finds a headcount satisfying the entered service-level and occupancy targets while accounting for shrinkage. Costs use hourly wage, hours per operating day, and workdays per month.

The chart compares estimated forecast-driven labor cost with staffing fixed at the peak projected headcount. The summary reports the recommended call-taker range, minimum modeled service level, peak occupancy, total modeled savings, holdout MAE, and MLflow run ID.

The monthly staffing calculation does not solve a shift schedule. It abstracts demand across configured operating hours and cannot directly represent hourly arrival peaks, breaks, agent availability, skills, queue state, overtime, or contractual scheduling rules.

- Update calls per request, handle time, shrinkage, wage, work hours, workdays, service target, answer time, and occupancy from verified operations data.
- A high monthly headcount or savings figure is not evidence of an operationally feasible roster.

6. Data meaning and support

NYC Open Data's 311 dataset contains service-request activity, not call-center ACD traffic or workforce records. Any call conversion and staffing assumptions are illustrative planning parameters until calibrated against appropriate operational systems.

For Parquet, data refresh, model, or tracking errors, review the R console and Python logs. Confirm required R packages, Python interpreter, dataset access, NYC Open Data availability, and MLflow write permissions. The repository README documents the shared model and tracking configuration.